

Ingénierie Dirigée par les Modèles : Transformations et Sémantiques

2SN-BML S7 2025–2026

Guillaume Dupont

`guillaume.dupont@toulouse-inp.fr`

Toulouse INP – ENSEEIHT



Cette présentation est sous licence Creative Common 4.0
BY-NC-SA
<https://creativecommons.org/licenses/by-nc-sa/4.0/>

Sommaire

- 1 Introduction
- 2 Transformations modèle/texte (M2T, T2M)
- 3 Transformations modèle-à-modèle (M2M)
- 4 Transfert sémantique, validation de transformation
- 5 Conclusion

Rappel

- ▶ L'**IDM** est un ensemble de **techniques** pour la conception, la maintenance, l'utilisation de logiciels, qui se concentre sur des *modèles* plutôt que sur des *implémentations*

Rappel

- ▶ L'**IDM** est un ensemble de **techniques** pour la conception, la maintenance, l'utilisation de logiciels, qui se concentre sur des *modèles* plutôt que sur des *implémentations*
- ▶ Un **modèle** est une représentation **abstraite** d'un objet destiné à répondre à des **questions**/des **besoins**

Rappel

- ▶ L'**IDM** est un ensemble de **techniques** pour la conception, la maintenance, l'utilisation de logiciels, qui se concentre sur des *modèles* plutôt que sur des *implémentations*
- ▶ Un **modèle** est une représentation **abstraite** d'un objet destiné à répondre à des **questions**/des **besoins**
- ▶ Un **modèle** est construit en suivant une **syntaxe abstraite** ou **méta-modèle**

Rappel

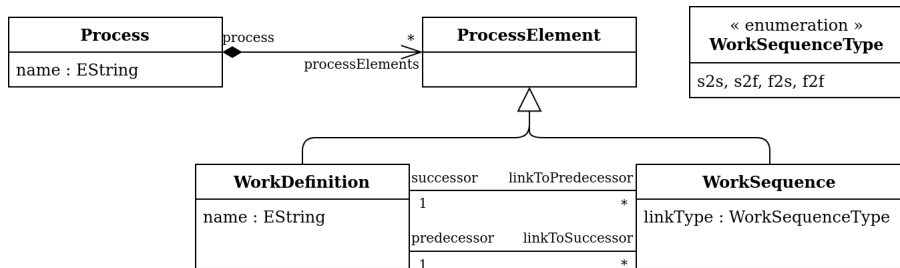
- ▶ L'**IDM** est un ensemble de **techniques** pour la conception, la maintenance, l'utilisation de logiciels, qui se concentre sur des *modèles* plutôt que sur des *implémentations*
- ▶ Un **modèle** est une représentation **abstraite** d'un objet destiné à répondre à des **questions**/des **besoins**
- ▶ Un **modèle** est construit en suivant une **syntaxe abstraite** ou **méta-modèle**
- ▶ Les **méta-modèles** sont construits en suivant le **méta-méta-modèle** (pour nous : EMOF)

Rappel

- ▶ L'**IDM** est un ensemble de **techniques** pour la conception, la maintenance, l'utilisation de logiciels, qui se concentre sur des *modèles* plutôt que sur des *implémentations*
- ▶ Un **modèle** est une représentation **abstraite** d'un objet destiné à répondre à des **questions**/des **besoins**
- ▶ Un **modèle** est construit en suivant une **syntaxe abstraite** ou **méta-modèle**
- ▶ Les **méta-modèles** sont construits en suivant le **méta-méta-modèle** (pour nous : EMOF)
- ▶ Les (méta-)modèles abstraient les **données**, les **transformations** abstraient les **processus**

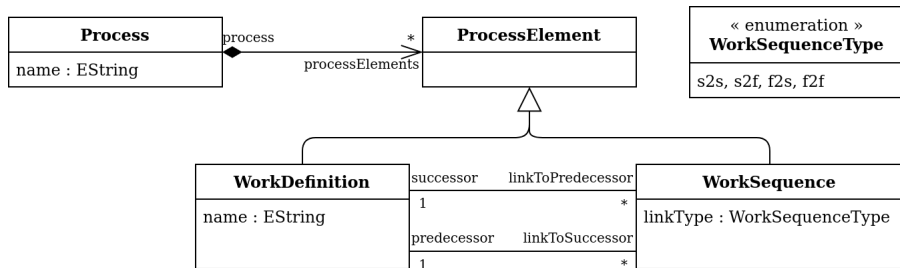
SimplePDL

Le méta-modèle fil rouge : SimplePDL



SimplePDL

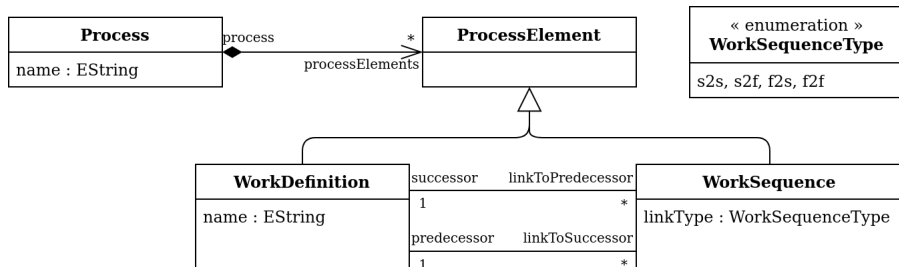
Le méta-modèle fil rouge : SimplePDL



- Représente des *tâches* (**WorkDefinition**) et des *dépendances* (**WorkSequence**) dans un *procédé* (**Process**)

SimplePDL

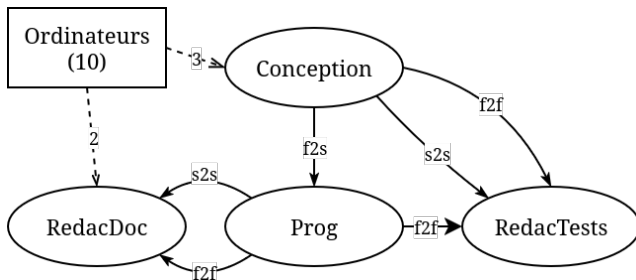
Le méta-modèle fil rouge : SimplePDL



- ▶ Représente des *tâches* (WorkDefinition) et des *dépendances* (WorkSequence) dans un *procédé* (Process)
- ▶ (il manque les ressources !)

Procédé "Développement"

Un des modèles fils rouges :



Motivation

Un modèle répondant à des **besoins précis**, est bon pour certaines choses, mais pas d'autres.

Motivation

Un modèle répondant à des **besoins précis**, est bon pour certaines choses, mais pas d'autres.

Exemple : un modèle SimplePDL admet-il des scénarios qui terminent ?

Motivation

Un modèle répondant à des **besoins précis**, est bon pour certaines choses, mais pas d'autres.

Exemple : un modèle SimplePDL admet-il des scénario qui terminent ?

On peut avoir besoin d'organiser les **données** d'un modèle autrement

Motivation

Un modèle répondant à des **besoins précis**, est bon pour certaines choses, mais pas d'autres.

Exemple : un modèle SimplePDL admet-il des scénarios qui terminent ?

On peut avoir besoin d'organiser les **données** d'un modèle autrement

Exemple : extraire une liste de tâches textuelle d'un modèle SimplePDL

Motivation

Un modèle répondant à des **besoins précis**, est bon pour certaines choses, mais pas d'autres.

Exemple : un modèle SimplePDL admet-il des scénario qui terminent ?

On peut avoir besoin d'organiser les **données** d'un modèle autrement

Exemple : extraire une liste de tâches textuelle d'un modèle SimplePDL

« Pourquoi ne pas utiliser Java ? Les modèles sont accessible grâce à EMF ! »

Motivation

Un modèle répondant à des **besoins précis**, est bon pour certaines choses, mais pas d'autres.

Exemple : un modèle SimplePDL admet-il des scénario qui terminent ?

On peut avoir besoin d'organiser les **données** d'un modèle autrement

Exemple : extraire une liste de tâches textuelle d'un modèle SimplePDL

« Pourquoi ne pas utiliser Java ? Les modèles sont accessible grâce à EMF ! »

⇒ c'est vrai, mais on perd l'aspect IDM

Motivation

Un modèle répondant à des **besoins précis**, est bon pour certaines choses, mais pas d'autres.

Exemple : un modèle SimplePDL admet-il des scénarios qui terminent ?

On peut avoir besoin d'organiser les **données** d'un modèle autrement

Exemple : extraire une liste de tâches textuelle d'un modèle SimplePDL

« Pourquoi ne pas utiliser Java ? Les modèles sont accessibles grâce à EMF ! »

⇒ c'est vrai, mais on perd l'aspect IDM

J'aimerais que les procédés soient aussi proches des modèles que possible (pas de boiler plate, que des éléments du méta-modèle...)

Motivation

Un modèle répondant à des **besoins précis**, est bon pour certaines choses, mais pas d'autres.

Exemple : un modèle SimplePDL admet-il des scénarios qui terminent ?

On peut avoir besoin d'organiser les **données** d'un modèle autrement

Exemple : extraire une liste de tâches textuelle d'un modèle SimplePDL

« Pourquoi ne pas utiliser Java ? Les modèles sont accessibles grâce à EMF ! »

⇒ c'est vrai, mais on perd l'aspect IDM

J'aimerais que les procédés soient aussi proches des modèles que possible (pas de boiler plate, que des éléments du méta-modèle...)

⇒ les transformations

Transformations

Définition

Une **transformation** est tout procédé, fonction, algorithme qui admet des modèles en entrée et produit des modèles en sortie.

Transformations

Définition

Une **transformation** est tout procédé, fonction, algorithme qui admet des modèles en entrée et produit des modèles en sortie.

Nous verrons deux grandes classes de transformation :

Transformations

Définition

Une **transformation** est tout procédé, fonction, algorithme qui admet des modèles en entrée et produit des modèles en sortie.

Nous verrons deux grandes classes de transformation :

- Les transformations de modèle à **texte** (M2T)

Transformations

Définition

Une **transformation** est tout procédé, fonction, algorithme qui admet des modèles en entrée et produit des modèles en sortie.

Nous verrons deux grandes classes de transformation :

- ▶ Les transformations de modèle à **texte** (M2T)
- ▶ Les transformations de modèle à **modèle** (M2M)

Transformations

Définition

Une **transformation** est tout procédé, fonction, algorithme qui admet des modèles en entrée et produit des modèles en sortie.

Nous verrons deux grandes classes de transformation :

- ▶ Les transformations de modèle à **texte** (M2T)
- ▶ Les transformations de modèle à **modèle** (M2M)

Questions :

Transformations

Définition

Une **transformation** est tout procédé, fonction, algorithme qui admet des modèles en entrée et produit des modèles en sortie.

Nous verrons deux grandes classes de transformation :

- ▶ Les transformations de modèle à **texte** (M2T)
- ▶ Les transformations de modèle à **modèle** (M2M)

Questions :

- ▶ Quel **formalisme** utiliser ?

Transformations

Définition

Une **transformation** est tout procédé, fonction, algorithme qui admet des modèles en entrée et produit des modèles en sortie.

Nous verrons deux grandes classes de transformation :

- ▶ Les transformations de modèle à **texte** (M2T)
- ▶ Les transformations de modèle à **modèle** (M2M)

Questions :

- ▶ Quel **formalisme** utiliser ?
- ▶ Comment **valider** une transformation ?

Sommaire

- 1 Introduction
- 2 Transformations modèle/texte (M2T, T2M)
 - Transformation M2T
 - Transformation T2M
- 3 Transformations modèle-à-modèle (M2M)
- 4 Transfert sémantique, validation de transformation
- 5 Conclusion

Présentation

Le but d'une transformation M2T est de transformer tout modèle conforme à un méta-modèle en **texte**.

Présentation

Le but d'une transformation M2T est de transformer tout modèle conforme à un méta-modèle en **texte**.

Plusieurs approches :

Présentation

Le but d'une transformation M2T est de transformer tout modèle conforme à un méta-modèle en **texte**.

Plusieurs approches :

- Approches par *patron* (*template*)

Présentation

Le but d'une transformation M2T est de transformer tout modèle conforme à un méta-modèle en **texte**.

Plusieurs approches :

- ▶ Approches par *patron* (*template*)
- ▶ Approches structurées (grammaires)

Patrons – Idée

Patrons

Un **patron** (ou *template*) est un texte brut décoré de directives requêtant un modèle ou exécutant du code.

Patrons – Idée

Patrons

Un **patron** (ou *template*) est un texte brut décoré de directives requêtant un modèle ou exécutant du code.

Exemples : Django, JSP, PHP, React (dans une certaine mesure)...

Patrons – Idée

Patrons

Un **patron** (ou *template*) est un texte brut décoré de directives requêtant un modèle ou exécutant du code.

Exemples : Django, JSP, PHP, React (dans une certaine mesure)...

Concrètement : du texte à trous, avec du code à exécuter (au lieu d'avoir du code avec du texte à afficher)

Patrons – Idée

Patrons

Un **patron** (ou *template*) est un texte brut décoré de directives requêtant un modèle ou exécutant du code.

Exemples : Django, JSP, PHP, React (dans une certaine mesure)...

Concrètement : du texte à trous, avec du code à exécuter (au lieu d'avoir du code avec du texte à afficher)

⇒ *le texte est central au patron*

Acceleo

La technologie EMF pour les transformations M2T que nous utiliserons est *Acceleo*

Acceleo

La technologie EMF pour les transformations M2T que nous utiliserons est *Acceleo*

Principe général :

Acceleo

La technologie EMF pour les transformations M2T que nous utiliserons est *Acceleo*

Principe général :

- texte + *balises* (délimitées par des crochets [])

Acceleo

La technologie EMF pour les transformations M2T que nous utiliserons est *Acceleo*

Principe général :

- ▶ texte + *balises* (délimitées par des crochets [])
- ▶ à l'exécution, le texte est retranscrit tel quel

Acceleo

La technologie EMF pour les transformations M2T que nous utiliserons est *Acceleo*

Principe général :

- ▶ texte + *balises* (délimitées par des crochets [])
- ▶ à l'exécution, le texte est retranscrit tel quel
- ▶ les directives sont exécutées et le résultat est retranscrit dans la sortie

Acceleo – Balises AQL

Les balises [... /] peuvent présenter du code *AQL* (Acceleo Query Language), similaire à OCL, qui est exécuté et retranscrit :

Acceleo – Balises AQL

Les balises [... /] peuvent présenter du code *AQL* (Acceleo Query Language), similaire à OCL, qui est exécuté et retranscrit :

```
Ceci est du texte.  
Qui contient [ 1 + 1 /] phrases.  
C'est [ if 5 * 13 <> 47 then 'trop_cool'  
      else 'pas_top' /]...
```

Acceleo – Balises AQL

Les balises [... /] peuvent présenter du code *AQL* (Acceleo Query Language), similaire à OCL, qui est exécuté et retranscrit :

```
Ceci est du texte.  
Qui contient [ 1 + 1 /] phrases.  
C'est [ if 5 * 13 <> 47 then 'trop_cool'  
      else 'pas_top' /]...
```

⇒

```
Ceci est du texte.  
Qui contient 2 phrases.  
C'est trop cool...
```

Acceleo – Balises AQL

Les balises [... /] peuvent présenter du code *AQL* (Acceleo Query Language), similaire à OCL, qui est exécuté et retranscrit :

```
Ceci est du texte.  
Qui contient [ 1 + 1 /] phrases.  
C'est [ if 5 * 13 <> 47 then 'trop_cool'  
      else 'pas_top' /]...
```

⇒

```
Ceci est du texte.  
Qui contient 2 phrases.  
C'est trop cool...
```

Le code AQL peut bien sûr faire référence au méta-modèle en entrée !

Acceleo – Balises AQL

Les balises [... /] peuvent présenter du code *AQL* (Acceleo Query Language), similaire à OCL, qui est exécuté et retranscrit :

```
Ceci est du texte.  
Qui contient [ 1 + 1 /] phrases.  
C'est [ if 5 * 13 <> 47 then 'trop_cool'  
      else 'pas_top' /]...
```

⇒

```
Ceci est du texte.  
Qui contient 2 phrases.  
C'est trop cool...
```

Le code AQL peut bien sûr faire référence au méta-modèle en entrée !

```
Le nom de la WD est "[ wd.name /]".
```

Acceleo – Balises AQL

Les balises [... /] peuvent présenter du code *AQL* (Acceleo Query Language), similaire à OCL, qui est exécuté et retranscrit :

```
Ceci est du texte.  
Qui contient [ 1 + 1 /] phrases.  
C'est [ if 5 * 13 <> 47 then 'trop_cool'  
      else 'pas_top' /]...
```

⇒

```
Ceci est du texte.  
Qui contient 2 phrases.  
C'est trop cool...
```

Le code AQL peut bien sûr faire référence au méta-modèle en entrée !

```
Le nom de la WD est "[ wd.name /]".
```

⇒

```
Le nom de la WD est "Conception".
```

Acceleo – Directives

Acceleo propose des directives conditionnelles et des boucles type *foreach* :

Acceleo – Directives

Acceleo propose des directives conditionnelles et des boucles type *foreach* :

```
[for (ws | wd.linksToSuccessors)]  
WS sortante de type [if ws.linkType =  
    #s2s or ws.linkType = #s2f]  
start to machin  
[else]  
finish to truc  
[/if]  
[/for]
```


Acceleo – Directives

Acceleo propose des directives conditionnelles et des boucles type *foreach* :

```
[for (ws | wd.linksToSuccessors)]  
WS sortante de type [if ws.linkType =  
    #s2s or ws.linkType = #s2f]  
start to machin  
[else]  
finish to truc  
[/if]  
[/for]
```

⇒

```
WS sortante de type finish to truc  
WS sortante de type start to machin  
WS sortante de type finish to truc
```

Acceleo – Directives

Acceleo propose des directives conditionnelles et des boucles type *foreach* :

```
[for (ws | wd.linksToSuccessors)]  
WS sortante de type [if ws.linkType =  
    #s2s or ws.linkType = #s2f]  
start to machin  
[else]  
finish to truc  
[/if]  
[/for]
```

⇒

```
WS sortante de type finish to truc  
WS sortante de type start to machin  
WS sortante de type finish to truc
```

Les directives Acceleo mangent des espaces et des retours à la ligne pour clarifier l'écriture

Acceleo – Template et Queries

Acceleo permet la définition de sous-programmes : *templates* pour afficher du texte, *queries* pour effectuer un calcul

Acceleo – Template et Queries

Acceleo permet la définition de sous-programmes : *templates* pour afficher du texte, *queries* pour effectuer un calcul

```
[query nbWS(wd : WorkDefinition) : Integer =  
    wd.linksToSuccessors.size() + wd.  
        linksToPredecessors.size()  
/]  
  
[template WDtoNb(wd : WorkDefinition)]  
La WD [wd.name /] a [wd.nbWS() /] relations.  
[/template]  
  
...  
[for (wd | process.workDefinitions)]  
[wd.WDtoNb() /]  
[/for]
```

Acceleo – Template et Queries

Acceleo permet la définition de sous-programmes : *templates* pour afficher du texte, *queries* pour effectuer un calcul

```
[query nbWS(wd : WorkDefinition) : Integer =  
    wd.linksToSuccessors.size() + wd.  
        linksToPredecessors.size()  
/]  
  
[template WDtoNb(wd : WorkDefinition)]  
La WD [wd.name /] a [wd.nbWS() /] relations.  
[/template]  
  
...  
[for (wd | process.workDefinitions)]  
[wd.WDtoNb() /]  
[/for]
```

⇒

```
La WD Conception a 3 relations.  
La WD RedacDoc a 2 relations.  
La WD Prog a 4 relations.  
La WD RedacTests a 3 relations.
```

Acceleo – Autres directives

On demande à Acceleo de créer un fichier avec **file**. Les templates et queries sont dans un *module*, qui contient une template *principale*

Acceleo – Autres directives

On demande à Acceleo de créer un fichier avec **file**. Les templates et queries sont dans un *module*, qui contient une template *principale*

```
[comment @main /]  
[template public spdLToText(p : Process)]  
[file (p.name + '.txt', overwrite, 'UTF-8')]  
Contenu de [p.name /] :  
[for (pe | p.processElements)]  
...  
[/for]  
[/file]  
[/template]
```

Acceleo – Autres directives

On demande à Acceleo de créer un fichier avec **file**. Les templates et queries sont dans un *module*, qui contient une template *principale*

```
[comment @main /]  
[template public spdLToText(p : Process)]  
[file (p.name + '.txt', overwrite, 'UTF-8')]  
Contenu de [p.name /] :  
[for (pe | p.processElements)]  
...  
[/for]  
[/file]  
[/template]
```

⇒

Contenu de Développement :
...

Sens inverse ?

On peut transformer du texte en modèle grâce à une **grammaire** (cf. UE/matière TDL)

Sens inverse ?

On peut transformer du texte en modèle grâce à une **grammaire** (cf. UE/matière TDL)

Une grammaire se compose d'un ensemble de *règles* qui expliquent comment former des *phrases*

Sens inverse ?

On peut transformer du texte en modèle grâce à une **grammaire** (cf. UE/matière TDL)

Une grammaire se compose d'un ensemble de *règles* qui expliquent comment former des *phrases*

On peut parler de transformation T2M, ou plutôt de **syntaxe concrète textuelle**

Sens inverse ?

On peut transformer du texte en modèle grâce à une **grammaire** (cf. UE/matière TDL)

Une grammaire se compose d'un ensemble de *règles* qui expliquent comment former des *phrases*

On peut parler de transformation T2M, ou plutôt de **syntaxe concrète textuelle**

En ce qui nous concerne, nous utiliserons Xtext (basé sur ANTLR)

Xtext – Règles de base

Une règle est associée à un non-terminal et se compose de base d'une séquence de terminaux ou de non terminaux

Xtext – Règles de base

Une règle est associée à un non-terminal et se compose de base d'une séquence de terminaux ou de non terminaux

Les terminaux peuvent-être des mots (pour faire des mots-clefs) ou des classes de mots

Xtext – Règles de base

Une règle est associée à un non-terminal et se compose de base d'une séquence de terminaux ou de non terminaux

Les terminaux peuvent-être des mots (pour faire des mots-clefs) ou des classes de mots

Process:

```
'process' STRING '{'  
    WorkDefinition  
'}';
```

WorkDefinition:

```
'wd' ID ';;'
```

Xtext – Règles de base

Une règle est associée à un non-terminal et se compose de base d'une séquence de terminaux ou de non terminaux

Les terminaux peuvent-être des mots (pour faire des mots-clefs) ou des classes de mots

Process:

```
'process' STRING '{'  
  WorkDefinition  
'}';
```

WorkDefinition:

```
'wd' ID ';;';
```

⇒

```
process "Mon_procédé" {  
  wd MaWD;  
}
```


Xtext – Règles de base

Une règle est associée à un non-terminal et se compose de base d'une séquence de terminaux ou de non terminaux

Les terminaux peuvent-être des mots (pour faire des mots-clefs) ou des classes de mots

Process:

```
'process' STRING '{'  
  WorkDefinition  
'}';
```

WorkDefinition:

```
'wd' ID ';;';
```

⇒

```
process "Mon_procédé" {  
  wd MaWD;  
}
```

La présence d'un non-terminal est un "appel" de règle.

Xtext – Choix, arité

Le `|` représente l'**alternative**, le `*` représente la **répétition** 0 ou plusieurs fois, le `?` l'**optionnel** et le `+` la répétition 1 ou plusieurs fois.

Xtext – Choix, arité

Le `|` représente l'**alternative**, le `*` représente la **répétition** 0 ou plusieurs fois, le `?` l'**optionnel** et le `+` la répétition 1 ou plusieurs fois.

Process:

```
'process' STRING '{'  
    ProcessElement+  
'}';
```

ProcessElement: WorkDefinition |
 WorkSequence;

WorkDefinition:

```
'wd' (ID (',' ID)*) ';' ;
```

WorkSequence;

```
'ws' ID 'to'? ID ';' ;
```

Xtext – Choix, arité

Le | représente l'**alternative**, le * représente la **répétition** 0 ou plusieurs fois, le ? l'**optionnel** et le + la répétition 1 ou plusieurs fois.

```

Process:
  'process' STRING '{'
    ProcessElement+
  '}' ;

ProcessElement: WorkDefinition |
  WorkSequence;

WorkDefinition:
  'wd' (ID (',' ID)*) ';' ;

WorkSequence;
  'ws' ID 'to'? ID ';' ;

```

⇒

```

process "Mon_procédé" {
  wd A, B, C;
  wd Z;
  ws A B;
  ws Z to C;
}

```

Correspondance grammaire–méta-modèle

Une grammaire *Xtext* est associée à un méta-modèle suivant des règles précises.

Correspondance grammaire–méta-modèle

Une grammaire *Xtext* est associée à un méta-modèle suivant des règles précises.

On peut enrichir le méta-modèle avec des *attributs* directement dans la grammaire !

Correspondance grammaire–méta-modèle

Une grammaire Xtext est associée à un méta-modèle suivant des règles précises.

On peut enrichir le méta-modèle avec des *attributs* directement dans la grammaire !

Process:

```
'process' name=STRING '{'  
    processElements+=ProcessElement+  
'}';
```

ProcessElement: WorkDefinition |
 WorkSequence;

WorkDefinition:

```
'wd' name=ID ';;';
```

WorkSequence;

```
'ws' linkType=LinkType ';;'
```

Correspondance grammaire-méta-modèle

Une grammaire Xtext est associée à un méta-modèle suivant des règles précises.

On peut enrichir le méta-modèle avec des *attributs* directement dans la grammaire !

Process:

```
'process' name=STRING '{'  
  processElements+=ProcessElement+  
'}';
```

ProcessElement: WorkDefinition |
WorkSequence;

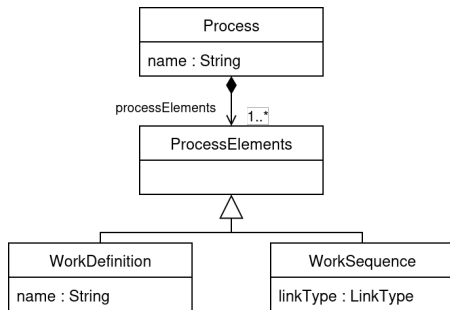
WorkDefinition:

```
'wd' name=ID ';;';
```

WorkSequence;

```
'ws' linkType=LinkType ';;'
```

⇒



Références

Grâce au nom d'attribut spécial *name* on peut faire référence à un objet créé par une règle plutôt que d'appeler la règle

Références

Grâce au nom d'attribut spécial *name* on peut faire référence à un objet créé par une règle plutôt que d'appeler la règle

```
WorkDefinition:
  'wd' name=ID ' ';

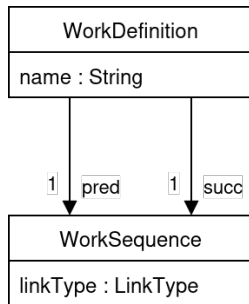
WorkSequence;
  'ws' pred=[WorkDefinition]
        linkType=LinkType
        succ=[WorkDefinition] ' ';
```

Références

Grâce au nom d'attribut spécial *name* on peut faire référence à un objet créé par une règle plutôt que d'appeler la règle

```
WorkDefinition:  
  'wd' name=ID ';;'  
  
WorkSequence;  
  'ws' pred=[WorkDefinition]  
        linkType=LinkType  
        succ=[WorkDefinition] ';;';
```

⇒

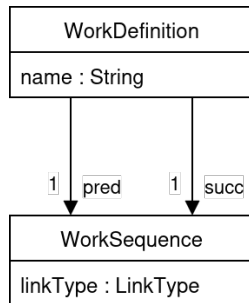


Références

Grâce au nom d'attribut spécial *name* on peut faire référence à un objet créé par une règle plutôt que d'appeler la règle

```
WorkDefinition:  
  'wd' name=ID ';;'  
  
WorkSequence;  
  'ws' pred=[WorkDefinition]  
        linkType=LinkType  
        succ=[WorkDefinition] ';;';
```

⇒



Remarque : pas de possibilité de faire des *opposit* en Xtext :(

Xtext – Remarques

Xtext permet de créer une syntaxe concrète textuelle pour un méta-modèle :

Xtext – Remarques

Xtext permet de créer une syntaxe concrète textuelle pour un méta-modèle :

- ▶ permet de transformer du texte en modèle

Xtext – Remarques

Xtext permet de créer une syntaxe concrète textuelle pour un méta-modèle :

- ▶ permet de transformer du texte en modèle
- ▶ crée un éditeur avec coloration syntaxique

Xtext – Remarques

Xtext permet de créer une syntaxe concrète textuelle pour un méta-modèle :

- ▶ permet de transformer du texte en modèle
- ▶ crée un éditeur avec coloration syntaxique

Xtext associe une grammaire à un méta-modèle :

Xtext – Remarques

Xtext permet de créer une syntaxe concrète textuelle pour un méta-modèle :

- ▶ permet de transformer du texte en modèle
- ▶ crée un éditeur avec coloration syntaxique

Xtext associe une grammaire à un méta-modèle :

- ▶ en général, on laisse à Xtext ce méta-modèle proche de la grammaire, et on fait une transformation (M2M) des modèles de ce méta-modèle vers le nôtre

Xtext – Remarques

Xtext permet de créer une syntaxe concrète textuelle pour un méta-modèle :

- ▶ permet de transformer du texte en modèle
- ▶ crée un éditeur avec coloration syntaxique

Xtext associe une grammaire à un méta-modèle :

- ▶ en général, on laisse à Xtext ce méta-modèle proche de la grammaire, et on fait une transformation (M2M) des modèles de ce méta-modèle vers le nôtre
(plus simple que d'essayer de coller une grammaire à MM pas fait pour)

Xtext – Remarques

Xtext permet de créer une syntaxe concrète textuelle pour un méta-modèle :

- ▶ permet de transformer du texte en modèle
- ▶ crée un éditeur avec coloration syntaxique

Xtext associe une grammaire à un méta-modèle :

- ▶ en général, on laisse à Xtext ce méta-modèle proche de la grammaire, et on fait une transformation (M2M) des modèles de ce méta-modèle vers le nôtre
(plus simple que d'essayer de coller une grammaire à MM pas fait pour)

Il y a plein de features intéressantes à Xtext non abordées ici mais explicitées dans la doc !

Sommaire

- 1 Introduction
- 2 Transformations modèle/texte (M2T, T2M)
- 3 Transformations modèle-à-modèle (M2M)**
- 4 Transfert sémantique, validation de transformation
- 5 Conclusion

Présentation

Une transformation de modèle à modèle est essentiellement une transformation de *graphe*

Présentation

Une transformation de modèle à modèle est essentiellement une transformation de *graphe* $\Rightarrow$ *du diagramme d'objet si on veut*

Présentation

Une transformation de modèle à modèle est essentiellement une transformation de *graphe* $\Rightarrow$ *du diagramme d'objet si on veut*

L'approche la plus simple pour écrire une transformation M2M est par un ensemble de *règles* : *Entrees* $\longrightarrow$ *Sorties*

Présentation

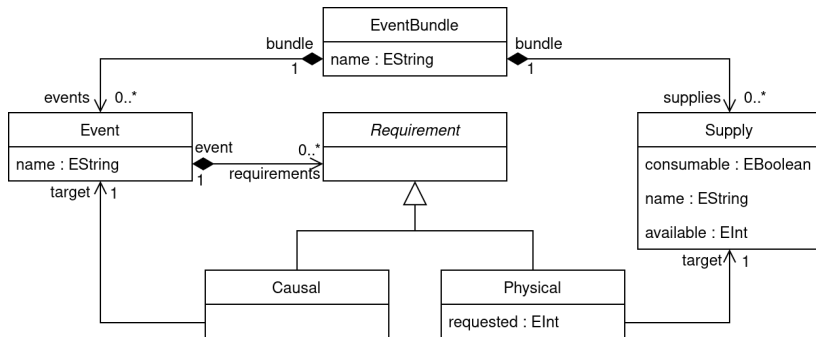
Une transformation de modèle à modèle est essentiellement une transformation de *graphe* $\Rightarrow$ *du diagramme d'objet si on veut*

L'approche la plus simple pour écrire une transformation M2M est par un ensemble de *règles* : *Entrees* $\longrightarrow$ *Sorties*

Dans la pratique, nous utiliserons la technologie *ATL*, un langage déclaratif permettant de définir un ensemble de règles

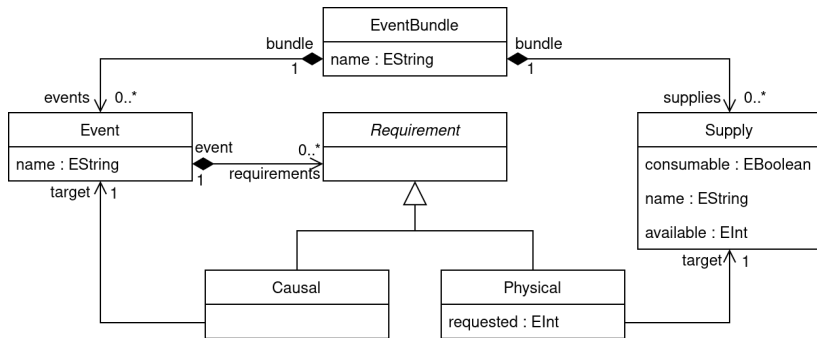
Modèle fil rouge : TASKMASTER

Dans la suite, on utilisera le MM TASKMASTER :



Modèle fil rouge : TASKMASTER

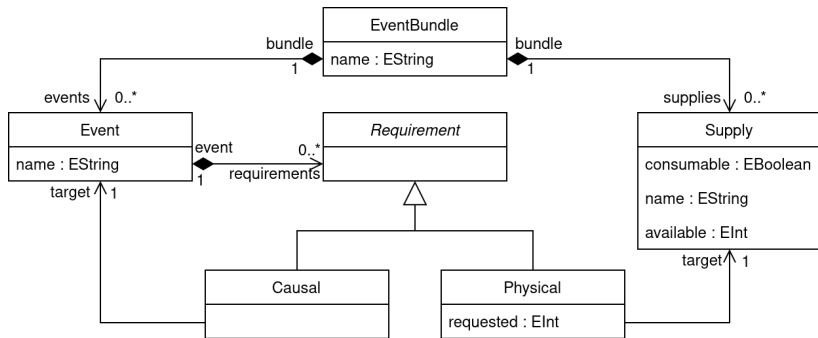
Dans la suite, on utilisera le MM TASKMASTER :



Ce MM permet de représenter des événements avec des dépendances entre eux, et des dépendances à des ressources

Modèle fil rouge : TASKMASTER

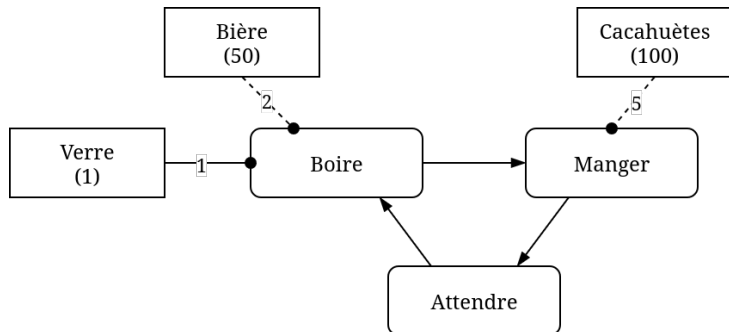
Dans la suite, on utilisera le MM TASKMASTER :



Ce MM permet de représenter des événements avec des dépendances entre eux, et des dépendances à des ressources

L'idée est de transformer un modèle SIMPLEPDL en modèle TASKMASTER

TASKMASTER – Exemple



ATL – Module

Une transformation ATL est définie dans un *module* qui définit les entrées et sorties de la transformation

ATL – Module

Une transformation ATL est définie dans un *module* qui définit les entrées et sorties de la transformation

```
module MaTransfo;  
create OUT1: MM1, OUT2: MM2, ... from IN1 : MM1, IN2 : MM2, ... :
```

ATL – Module

Une transformation ATL est définie dans un *module* qui définit les entrées et sorties de la transformation

```
module MaTransfo;  
create OUT1: MM1, OUT2: MM2, ... from IN1 : MM1, IN2 : MM2, ... :
```

Le nom de chaque entrée/sortie permet de faire référence à un élément du MM associé (grâce au !)

ATL – Module

Une transformation ATL est définie dans un *module* qui définit les entrées et sorties de la transformation

```
module MaTransfo;
create OUT1: MM1, OUT2: MM2, ... from IN1 : MM1, IN2 : MM2, ... :
```

Le nom de chaque entrée/sortie permet de faire référence à un élément du MM associé (grâce au !)

```
module SPDL2TM;
create OUT : TaskMaster from IN : SimplePDL :

...
TaskMaster!EventBundle.name
OUT!EventBundle.name -- Équivalent
...
```


ATL – Helpers

Comme en OCL, on peut définir des fonctions et potentiellement les attacher à un contexte

ATL – Helpers

Comme en OCL, on peut définir des fonctions et potentiellement les attacher à un contexte

```
helper def: nombreMagique(p : Integer) : Integer = p * 2;  
helper context TaskMaster!EventBundle def: nombreEvents : Integer =  
    self.events->size();  
  
...  
thisModule.nombreMagique(42) -- Helper "global"  
myBundle.nombreEvents -- Helper attaché à un contexte
```

ATL – Règles

Une règle mappe un ou plusieurs objets du MM d'entrée à un ou plusieurs objets du MM de sortie

ATL – Règles

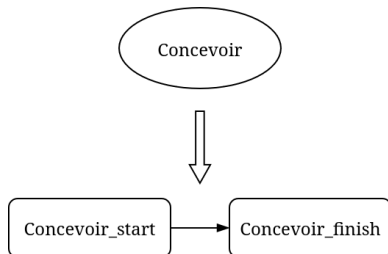
Une règle mappe un ou plusieurs objets du MM d'entrée à un ou plusieurs objets du MM de sortie

```
rule WD2Events {  
  from wd: SimplePDL!WorkDefinition  
  to   estart: TaskMaster!Event(  
        name <- wd.name + '_start'  
      ),  
      efinish: TaskMaster!Event(  
        name <- wd.name + '_finish'  
      ),  
      finish_start: TaskMaster!Causal(  
        event <- efinish,  
        target <- estart  
      )  
}
```

ATL – Règles

Une règle mappe un ou plusieurs objets du MM d'entrée à un ou plusieurs objets du MM de sortie

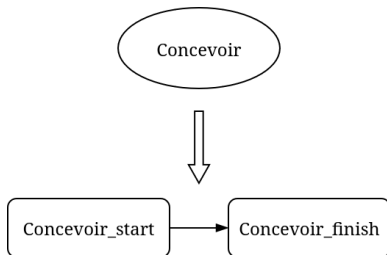
```
rule WD2Events {  
  from wd: SimplePDL!WorkDefinition  
  to   estart: TaskMaster!Event(  
    name <- wd.name + '_start'  
  ),  
    efinish: TaskMaster!Event(  
    name <- wd.name + '_finish'  
  ),  
    finish_start: TaskMaster!Causal(  
    event <- efinish,  
    target <- estart  
  )  
}
```



ATL – Règles

Une règle mappe un ou plusieurs objets du MM d'entrée à un ou plusieurs objets du MM de sortie

```
rule WD2Events {  
  from wd: SimplePDL!WorkDefinition  
  to   estart: TaskMaster!Event(  
        name <- wd.name + '_start'  
      ),  
      efinish: TaskMaster!Event(  
        name <- wd.name + '_finish'  
      ),  
      finish_start: TaskMaster!Causal(  
        event <- efinish,  
        target <- estart  
      )  
}
```



L'affectation d'attribut se fait avec des expressions OCL

ATL – Résolution

On peut aller chercher un objet créé par une autre règle par nom (résolution)

ATL – Résolution

On peut aller chercher un objet créé par une autre règle par nom (résolution)

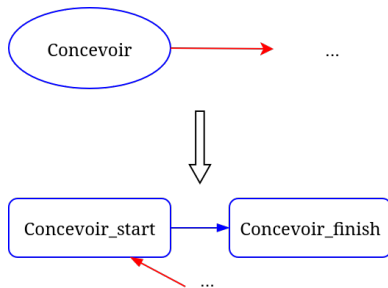
```
rule WD2Events {  
  from wd: SimplePDL!WorkDefinition  
  to   estart: TaskMaster!Event(  
    name <- wd.name + '_start'  
  ),  
  ...  
}  
  
rule WS2Causal {  
  from ws: SimplePDL!WorkSequence  
  to   rel: TaskMaster!Causal(  
    event <- thisModule.  
      resolveTemp(  
        ws.successor, 'estart'  
      )  
    ...  
  )  
}
```


ATL – Résolution

On peut aller chercher un objet créé par une autre règle par nom (résolution)

```
rule WD2Events {
  from wd: SimplePDL!WorkDefinition
  to estart: TaskMaster!Event(
    name <- wd.name + '_start'
  ),
  ...
}

rule WS2Causal {
  from ws: SimplePDL!WorkSequence
  to rel: TaskMaster!Causal(
    event <- thisModule.
      resolveTemp(
        ws.successor, 'estart')
    ...
  )
}
```



ATL – Résolution

Lorsque l'objet qu'on veut résoudre est inambigu, on peut l'appeler directement

ATL – Résolution

Lorsque l'objet qu'on veut résoudre est inambigu, on peut l'appeler directement

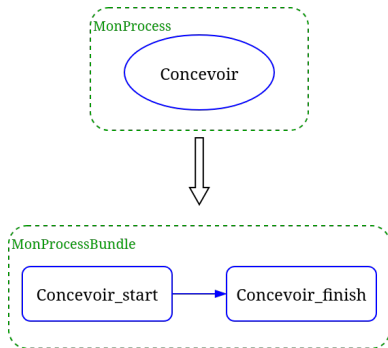
```
rule Process2Bundle {  
  from p: SimplePDL!Process  
  to   b: TaskMaster!EventBundle(  
    name <- p.name + 'Bundle'  
  )  
}  
  
rule WD2Event {  
  from wd: SimplePDL!WorkDefinition,  
  to   estart: TaskMaster!Event(  
    wd.process, ...),  
    ...  
}
```

ATL – Résolution

Lorsque l'objet qu'on veut résoudre est inambigu, on peut l'appeler directement

```
rule Process2Bundle {
  from p: SimplePDL!Process
  to b: TaskMaster!EventBundle(
    name <- p.name + 'Bundle')
}

rule WD2Event {
  from wd: SimplePDL!WorkDefinition,
  to estart: TaskMaster!Event(
    wd.process, ...),
  ...
}
```



Exécution

ATL détermine tout seul l'ordre d'exécution des règles.

Exécution

ATL détermine tout seul l'ordre d'exécution des règles.

$\Rightarrow$ *visite MM entrée + graphe dépendance*

Exécution

ATL détermine tout seul l'ordre d'exécution des règles.

$\Rightarrow$ *visite MM entrée + graphe dépendance*

ATL maintient un modèle de traçabilité, qui stocke les liens entre MM d'entrée et MM de sortie (*on dit aussi modèle de **tissage***

Exécution

ATL détermine tout seul l'ordre d'exécution des règles.

$\Rightarrow$ *visite MM entrée + graphe dépendance*

ATL maintient un modèle de traçabilité, qui stocke les liens entre MM d'entrée et MM de sortie (*on dit aussi modèle de **tissage***

ATL a plein d'autres features à explorer dans la doc !

Sommaire

- 1 Introduction
- 2 Transformations modèle/texte (M2T, T2M)
- 3 Transformations modèle-à-modèle (M2M)
- 4 Transfert sémantique, validation de transformation
 - Sémantique statique
 - Sémantique dynamique
- 5 Conclusion

Validation de Transformation

L'ordre dans lequel on exécute des règles de transformation n'est généralement pas connu, tester directement est donc un peu difficile.

Validation de Transformation

L'ordre dans lequel on exécute des règles de transformation n'est généralement pas connu, tester directement est donc un peu difficile. Plutôt que de faire du test direct, on se repose plutôt sur tester des propriétés.

Validation de Transformation

L'ordre dans lequel on exécute des règles de transformation n'est généralement pas connu, tester directement est donc un peu difficile. Plutôt que de faire du test direct, on se repose plutôt sur tester des propriétés.

Transfert sémantique : une transformation préserve généralement certains aspects sémantiques du modèle en entrée ; un modèle en entrée valide et respectant certains propriétés donne un modèle en sortie valide avec certaine propriétés.

Validation de Transformation

L'ordre dans lequel on exécute des règles de transformation n'est généralement pas connu, tester directement est donc un peu difficile. Plutôt que de faire du test direct, on se repose plutôt sur tester des propriétés.

Transfert sémantique : une transformation préserve généralement certains aspects sémantiques du modèle en entrée ; un modèle en entrée valide et respectant certains propriétés donne un modèle en sortie valide avec certaine propriétés.

$\Rightarrow$ *contrats*

Validation de Transformation

L'ordre dans lequel on exécute des règles de transformation n'est généralement pas connu, tester directement est donc un peu difficile. Plutôt que de faire du test direct, on se repose plutôt sur tester des propriétés.

Transfert sémantique : une transformation préserve généralement certains aspects sémantiques du modèle en entrée ; un modèle en entrée valide et respectant certains propriétés donne un modèle en sortie valide avec certaine propriétés.

$\Rightarrow$ *contrats*

Comment vérifier les propriétés en sortie ?

Validation de Transformation

L'ordre dans lequel on exécute des règles de transformation n'est généralement pas connu, tester directement est donc un peu difficile. Plutôt que de faire du test direct, on se repose plutôt sur tester des propriétés.

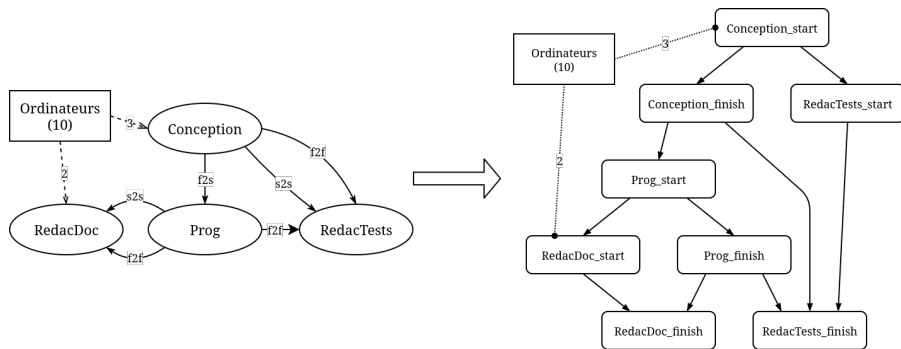
Transfert sémantique : une transformation préserve généralement certains aspects sémantiques du modèle en entrée ; un modèle en entrée valide et respectant certains propriétés donne un modèle en sortie valide avec certaine propriétés.

⇒ *contrats*

Comment vérifier les propriétés en sortie ?

⇒ OCL, autres outils spécifiques (LTL...), ...

Exemple : SIMPLEPDL vers TaskMaster



Objectif : écrire un OCL de *validation*

Exemple : SIMPLEPDL vers TaskMaster

"Il ne doit y avoir que des événements terminant en start ou finish"

Exemple : SIMPLEPDL vers TaskMaster

"Il ne doit y avoir que des événements terminant en start ou finish"

```
context EventBundle
inv NoForeignEvent:
  self.events->forall(e : Event |
    e.matches('.*_start') or e.matches('.*_finish'))
```

Exemple : SIMPLEPDL vers TaskMaster

"Il ne doit y avoir que des événements terminant en start ou finish"

```
context EventBundle
inv NoForeignEvent:
  self.events->forall(e : Event |
    e.matches('.*_start') or e.matches('.*_finish'))
```

"Tout événement finish est associé à un événement start"

Exemple : SIMPLEPDL vers TaskMaster

"Il ne doit y avoir que des événements terminant en start ou finish"

```
context EventBundle
inv NoForeignEvent:
  self.events->forall(e : Event |
    e.matches('.*_start') or e.matches('.*_finish'))
```

"Tout événement finish est associé à un événement start"

```
context Event
inv EventConsistency:
  self.name.matches('.*_finish') implies
    self.requirements->select(r | r.ocIsKind(Causal))
    ->collect(r | r.ocAsType(Causal).target)
    ->exists(e | e.matches('.*_start'))
```

Exemple : SIMPLEPDL vers TaskMaster

"Il ne doit y avoir que des événements terminant en start ou finish"

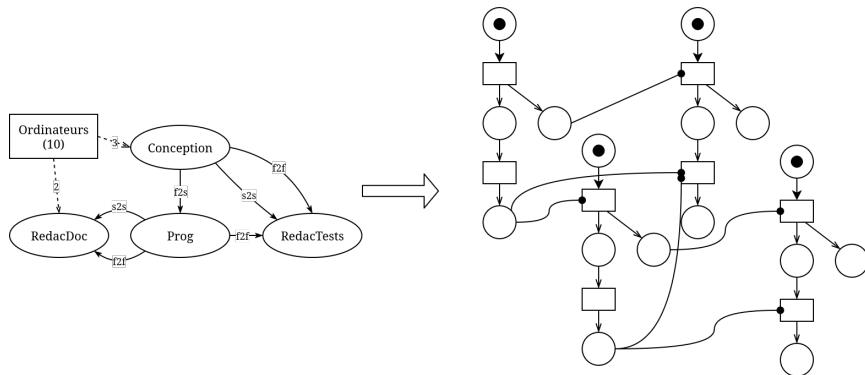
```
context EventBundle
inv NoForeignEvent:
  self.events->forall(e : Event |
    e.matches('.*_start') or e.matches('.*_finish'))
```

"Tout événement finish est associé à un événement start"

```
context Event
inv EventConsistency:
  self.name.matches('.*_finish') implies
    self.requirements->select(r | r.ocIsKind(Causal))
    ->collect(r | r.ocAsType(Causal).target)
    ->exists(e | e.matches('.*_start'))
```

...

SIMPLEPDL vers PETRINET



Exemple de questions de validation

- ▶ "Une tâche est soit non commencée, soit en cours, soit terminée"

Exemple de questions de validation

- ▶ "Une tâche est soit non commencée, soit en cours, soit terminée"
- ▶ "Une tâche commencée reste commencée pour toujours"

Exemple de questions de validation

- ▶ "Une tâche est soit non commencée, soit en cours, soit terminée"
- ▶ "Une tâche commencée reste commencée pour toujours"
- ▶ "Une tâche terminée reste terminée pour toujours"

Exemple de questions de validation

- ▶ "Une tâche est soit non commencée, soit en cours, soit terminée"
- ▶ "Une tâche commencée reste commencée pour toujours"
- ▶ "Une tâche terminée reste terminée pour toujours"
- ▶ "Une tâche terminée a commencé un jour"

Exemple de questions de validation

- ▶ "Une tâche est soit non commencée, soit en cours, soit terminée"
- ▶ "Une tâche commencée reste commencée pour toujours"
- ▶ "Une tâche terminée reste terminée pour toujours"
- ▶ "Une tâche terminée a commencé un jour"
- ▶ "Initialement, aucune tâche n'est commencé"

Exemple de questions de validation

- ▶ "Une tâche est soit non commencée, soit en cours, soit terminée"
- ▶ "Une tâche commencée reste commencée pour toujours"
- ▶ "Une tâche terminée reste terminée pour toujours"
- ▶ "Une tâche terminée a commencé un jour"
- ▶ "Initialement, aucune tâche n'est commencé"
- ▶ Et sur les ressources ?

Exemple de questions de validation

- ▶ "Une tâche est soit non commencée, soit en cours, soit terminée"
- ▶ "Une tâche commencée reste commencée pour toujours"
- ▶ "Une tâche terminée reste terminée pour toujours"
- ▶ "Une tâche terminée a commencé un jour"
- ▶ "Initialement, aucune tâche n'est commencé"
- ▶ Et sur les ressources ?

Des questionnements sur des *réseaux de Pétri* nécessitent des outils et des formalismes *adaptés*...

LTL – Propositions

On peut poser des questions sur un réseau de pétri de la forme "*place X a au moins/au plus/exactement n jetons*"

LTL – Propositions

On peut poser des questions sur un réseau de pétri de la forme "*place X a au moins/au plus/exactement n jetons*"

Propositions

Les *propositions* possibles sur un réseau de Pétri sont de la forme :

$$P ::= \top \mid \perp \mid \alpha = \alpha \mid \alpha \leq \alpha \mid \alpha < \alpha \mid \dots$$

Avec α les *termes* de la forme :

$$\alpha ::= \text{nom d'une place} \mid \text{nombre} \mid \alpha + \alpha \mid \alpha * \alpha$$

LTL – Propositions

On peut poser des questions sur un réseau de pétri de la forme "*place X a au moins/au plus/exactement n jetons*"

Propositions

Les *propositions* possibles sur un réseau de Pétri sont de la forme :

$$P ::= \top \mid \perp \mid \alpha = \alpha \mid \alpha \leq \alpha \mid \alpha < \alpha \mid \dots$$

Avec α les *termes* de la forme :

$$\alpha ::= \text{nom d'une place} \mid \text{nombre} \mid \alpha + \alpha \mid \alpha * \alpha$$

Par exemple : $A + B = 5$, $A * 2 + B * 3 \geq E + 6$, $\text{Concevoir_idle} = 1$, ...

LTL – Valuation

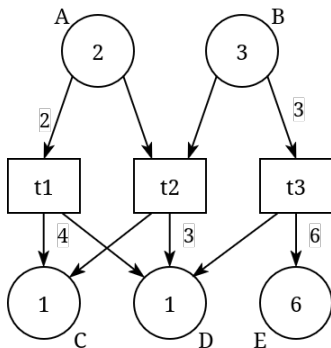
Valuation

Soit un réseau de Pétri $\mathcal{R}$ dans un état (=marquage) σ et une proposition P . La *valuation* de P dans $\mathcal{R}$ à l'état σ , noté $\llbracket P \rrbracket_{\mathcal{R}, \sigma}$ est la "valeur" de P lorsque l'on remplace chaque symbole par sa valeur.

LTL – Valuation

Valuation

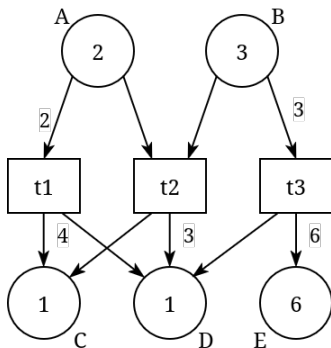
Soit un réseau de Pétri $\mathcal{R}$ dans un état (=marquage) σ et une proposition P . La *valuation* de P dans $\mathcal{R}$ à l'état σ , noté $\llbracket P \rrbracket_{\mathcal{R}, \sigma}$ est la "valeur" de P lorsque l'on remplace chaque symbole par sa valeur.



LTL – Valuation

Valuation

Soit un réseau de Pétri $\mathcal{R}$ dans un état (=marquage) σ et une proposition P . La *valuation* de P dans $\mathcal{R}$ à l'état σ , noté $\llbracket P \rrbracket_{\mathcal{R}, \sigma}$ est la "valeur" de P lorsque l'on remplace chaque symbole par sa valeur.

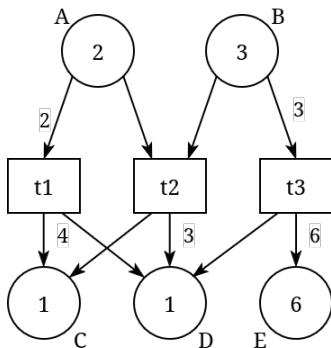


$$\blacktriangleright \llbracket A + B = 5 \rrbracket = \llbracket 2 + 3 = 5 \rrbracket = \llbracket 5 = 5 \rrbracket = \top$$

LTL – Valuation

Valuation

Soit un réseau de Pétri $\mathcal{R}$ dans un état (=marquage) σ et une proposition P . La *valuation* de P dans $\mathcal{R}$ à l'état σ , noté $\llbracket P \rrbracket_{\mathcal{R}, \sigma}$ est la "valeur" de P lorsque l'on remplace chaque symbole par sa valeur.

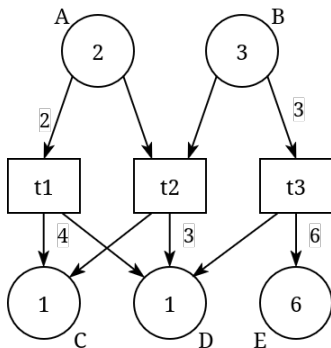


- ▶ $\llbracket A + B = 5 \rrbracket = \llbracket 2 + 3 = 5 \rrbracket = \llbracket 5 = 5 \rrbracket = \top$
- ▶ $\llbracket A * 2 + B * 3 \geq E + 6 \rrbracket = \llbracket 2 * 2 + 3 * 3 \geq 6 + 6 \rrbracket = \llbracket 11 \geq 12 \rrbracket = \perp$

LTL – Valuation

Valuation

Soit un réseau de Pétri $\mathcal{R}$ dans un état (=marquage) σ et une proposition P . La *valuation* de P dans $\mathcal{R}$ à l'état σ , noté $\llbracket P \rrbracket_{\mathcal{R}, \sigma}$ est la "valeur" de P lorsque l'on remplace chaque symbole par sa valeur.



- ▶ $\llbracket A + B = 5 \rrbracket = \llbracket 2 + 3 = 5 \rrbracket = \llbracket 5 = 5 \rrbracket = \top$
- ▶ $\llbracket A * 2 + B * 3 \geq E + 6 \rrbracket = \llbracket 2 * 2 + 3 * 3 \geq 6 + 6 \rrbracket = \llbracket 11 \geq 12 \rrbracket = \perp$
- ▶ $\llbracket \top \rrbracket = \top, \llbracket \perp \rrbracket = \perp$

LTL – Logique propositionnelle

On peut combiner des propositions avec des connecteurs logiques

LTL – Logique propositionnelle

On peut combiner des propositions avec des connecteurs logiques

Logique propositionnelle

Les *formules bien formées* de logique propositionnelle sur les réseaux de Pétri sont de la forme :

$$\varphi ::= P \mid \varphi \wedge \varphi \mid \varphi \vee \varphi \mid \varphi \Rightarrow \varphi \mid \varphi \Leftrightarrow \varphi \mid \neg \varphi$$

LTL – Logique propositionnelle

On peut combiner des propositions avec des connecteurs logiques

Logique propositionnelle

Les *formules bien formées* de logique propositionnelle sur les réseaux de Pétri sont de la forme :

$$\varphi ::= P \mid \varphi \wedge \varphi \mid \varphi \vee \varphi \mid \varphi \Rightarrow \varphi \mid \varphi \Leftrightarrow \varphi \mid \neg \varphi$$

Par exemple : $A = 1 \vee B = 1, D + C > 1 \Rightarrow E < 6$, etc.

LTL – Logique propositionnelle

On peut combiner des propositions avec des connecteurs logiques

Logique propositionnelle

Les *formules bien formées* de logique propositionnelle sur les réseaux de Pétri sont de la forme :

$$\varphi ::= P \mid \varphi \wedge \varphi \mid \varphi \vee \varphi \mid \varphi \Rightarrow \varphi \mid \varphi \Leftrightarrow \varphi \mid \neg \varphi$$

Par exemple : $A = 1 \vee B = 1, D + C > 1 \Rightarrow E < 6$, etc.

On propage la notion de valuation de proposition aux formules bien formées : $\llbracket \varphi \rrbracket_{\mathcal{R}, \sigma}$ est la "valeur" de φ pour le réseau $\mathcal{R}$ à l'état σ .

LTL – Sémantique propositionnelle

Validité (Sémantique)

Pour un réseau de Pétri donné $\mathcal{R}$ dans un état (=marquage) σ et une formule bien formée φ de logique propositionnelle, on dit que φ est **valide** pour $\mathcal{R}$ et σ et on note

$$\mathcal{R}, \sigma \models \varphi$$

$$\text{ssi } \llbracket \varphi \rrbracket_{\mathcal{R}, \sigma} = \top.$$

LTL – Sémantique propositionnelle

Validité (Sémantique)

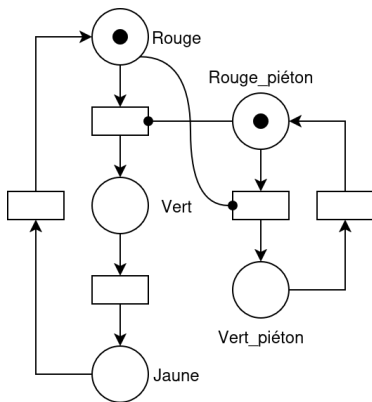
Pour un réseau de Pétri donné $\mathcal{R}$ dans un état (=marquage) σ et une formule bien formée φ de logique propositionnelle, on dit que φ est **valide** pour $\mathcal{R}$ et σ et on note

$$\mathcal{R}, \sigma \models \varphi$$

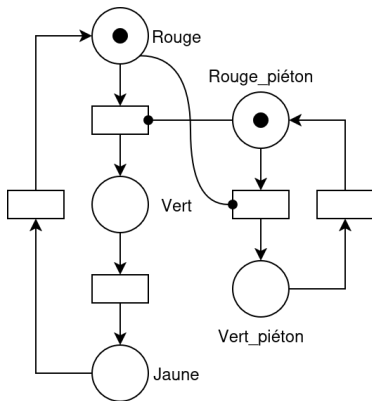
ssi $\llbracket \varphi \rrbracket_{\mathcal{R}, \sigma} = \top$.

Lorsque σ n'est pas précisé, il s'agit par défaut du **marquage initial** de $\mathcal{R}$

LTL – Exemple

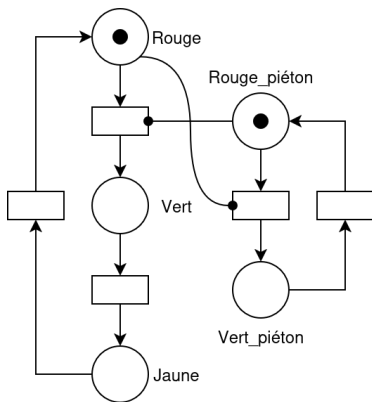


LTL – Exemple



► $\models \text{Rouge} + \text{Jaune} + \text{Vert} = 1$

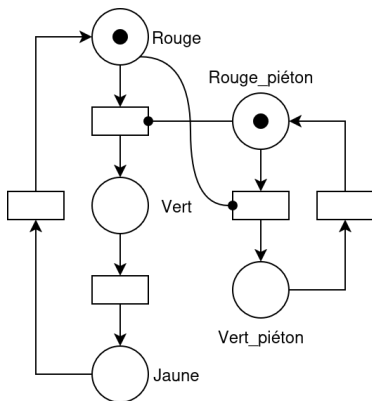
LTL – Exemple



► $\models \text{Rouge} + \text{Jaune} + \text{Vert} = 1$

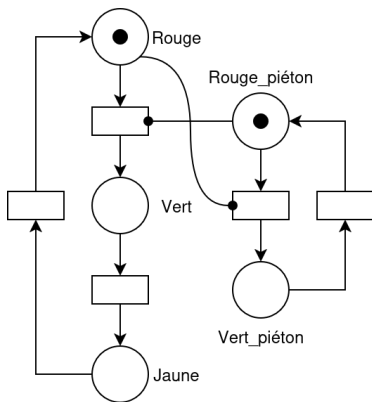
► $\models \text{Vert} > 0 \Rightarrow \text{Rouge_pieton} > 0$

LTL – Exemple



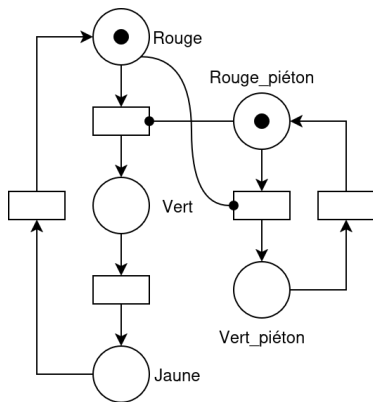
- ▶ $\models \text{Rouge} + \text{Jaune} + \text{Vert} = 1$
- ▶ $\models \text{Vert} > 0 \Rightarrow \text{Rouge_pieton} > 0$
- ▶ $\not\models \text{Vert_pieton} = 1$

LTL – Exemple



- ▶ $\models Rouge + Jaune + Vert = 1$
- ▶ $\models Vert > 0 \Rightarrow Rouge_pieton > 0$
- ▶ $\not\models Vert_pieton = 1$
- ▶ "Vert et vert_piéton n'ont jamais de jeton en même temps" ?

LTL – Exemple



- ▶ $\models Rouge + Jaune + Vert = 1$
- ▶ $\models Vert > 0 \Rightarrow Rouge_pieton > 0$
- ▶ $\not\models Vert_pieton = 1$
- ▶ "Vert et vert_piéton n'ont jamais de jeton en même temps" ?
- ▶ "Vert_piéton contiendra un jeton" ?

LTL – Modalités

Logique Temporelle Linéaire

On introduit deux opérateurs appelés *modalités* dans le langage des formules bien formées :

$$\varphi ::= \Box\varphi \mid \Diamond\varphi \mid \varphi \wedge \varphi \mid \varphi \vee \varphi \mid \neg\varphi \mid \varphi \Rightarrow \varphi \mid \varphi \Leftrightarrow \varphi$$

Le langage qu'on obtient est celui de la *logique temporelle linéaire*

LTL – Sémantique

Validité (Sémantique)

Pour un réseau $\mathcal{R}$ dans un état σ , on a $\mathcal{R}, \sigma \models \Box\varphi$ ($\Box\varphi$ est **valide**) ssi pour tout σ' **atteignable** depuis σ , on a $\mathcal{R}, \sigma' \models \varphi$ (incluant σ lui-même).

De manière duale, on a $\mathcal{R}, \sigma \models \Diamond\varphi$ ssi il existe σ' **atteignable** depuis σ tel que $\mathcal{R}, \sigma' \models \varphi$ (incluant σ lui-même).

LTL – Sémantique

Validité (Sémantique)

Pour un réseau $\mathcal{R}$ dans un état σ , on a $\mathcal{R}, \sigma \models \Box\varphi$ ($\Box\varphi$ est **valide**) ssi pour tout σ' **atteignable** depuis σ , on a $\mathcal{R}, \sigma' \models \varphi$ (incluant σ lui-même).

De manière duale, on a $\mathcal{R}, \sigma \models \Diamond\varphi$ ssi il existe σ' **atteignable** depuis σ tel que $\mathcal{R}, \sigma' \models \varphi$ (incluant σ lui-même).

Interprétation :

LTL – Sémantique

Validité (Sémantique)

Pour un réseau $\mathcal{R}$ dans un état σ , on a $\mathcal{R}, \sigma \models \Box\varphi$ ($\Box\varphi$ est **valide**) ssi pour tout σ' **atteignable** depuis σ , on a $\mathcal{R}, \sigma' \models \varphi$ (incluant σ lui-même).

De manière duale, on a $\mathcal{R}, \sigma \models \Diamond\varphi$ ssi il existe σ' **atteignable** depuis σ tel que $\mathcal{R}, \sigma' \models \varphi$ (incluant σ lui-même).

Interprétation :

- ▶ $\Box\varphi$ signifie " φ est vrai et le reste" (*toujours*)

LTL – Sémantique

Validité (Sémantique)

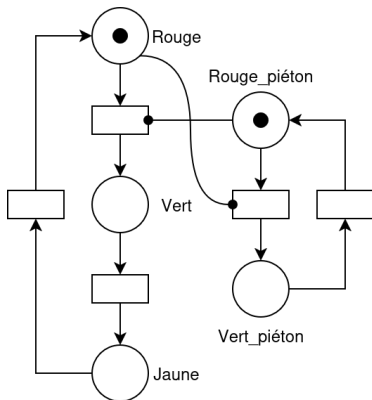
Pour un réseau $\mathcal{R}$ dans un état σ , on a $\mathcal{R}, \sigma \models \Box\varphi$ ($\Box\varphi$ est **valide**) ssi pour tout σ' **atteignable** depuis σ , on a $\mathcal{R}, \sigma' \models \varphi$ (incluant σ lui-même).

De manière duale, on a $\mathcal{R}, \sigma \models \Diamond\varphi$ ssi il existe σ' **atteignable** depuis σ tel que $\mathcal{R}, \sigma' \models \varphi$ (incluant σ lui-même).

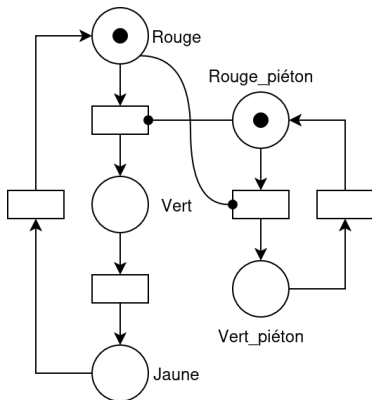
Interprétation :

- ▶ $\Box\varphi$ signifie " φ est vrai et le reste" (*toujours*)
- ▶ $\Diamond\varphi$ signifie " φ sera vrai un jour" (*un jour*)

LTL – Exemple avec modalités

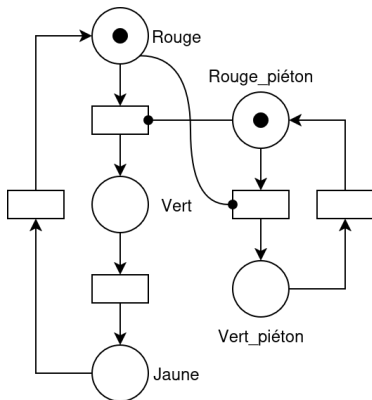


LTL – Exemple avec modalités



► $\models \Box(Rouge + Jaune + Vert = 1)$

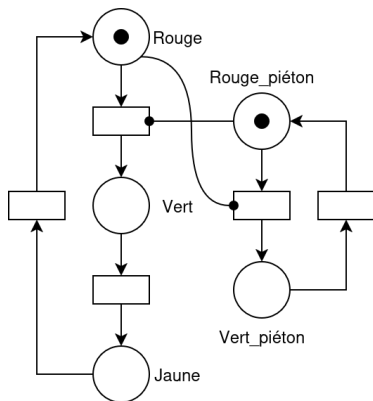
LTL – Exemple avec modalités



► $\models \Box(Rouge + Jaune + Vert = 1)$

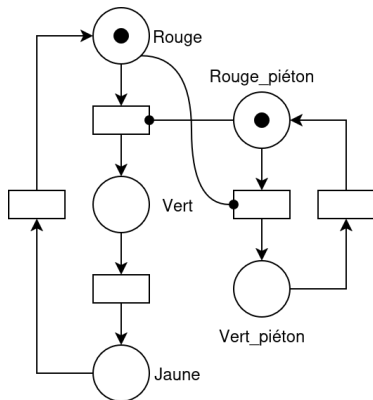
► $\models \Box(Vert > 0 \Rightarrow Rouge_pieton > 0)$

LTL – Exemple avec modalités



- ▶ $\models \Box(Rouge + Jaune + Vert = 1)$
- ▶ $\models \Box(Vert > 0 \Rightarrow Rouge_pieton > 0)$
- ▶ "Vert et vert_piéton n'ont jamais de jeton en même temps" ?
 $\models \neg \Diamond(Vert = 1 \wedge Vert_pieton = 1)$

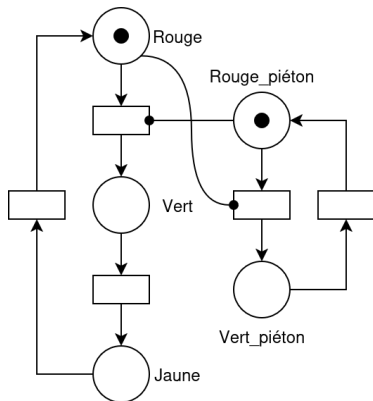
LTL – Exemple avec modalités



- ▶ $\models \Box(Rouge + Jaune + Vert = 1)$
- ▶ $\models \Box(Vert > 0 \Rightarrow Rouge_pieton > 0)$
- ▶ "Vert et vert_piéton n'ont jamais de jeton en même temps" ?
 $\models \neg \Diamond(Vert = 1 \wedge Vert_pieton = 1)$
- ▶ "Vert_piéton contiendra un jeton" ?
 $\models \Diamond(Vert_pieton = 1)$

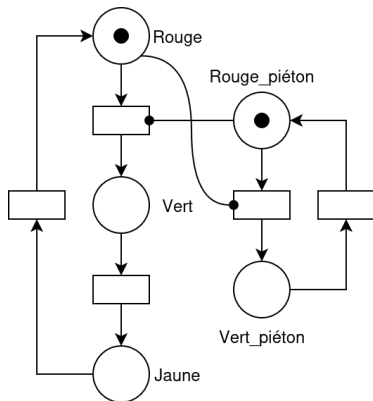
LTL – Exemple avec modalités

Attention à la combinaison de modalités !



LTL – Exemple avec modalités

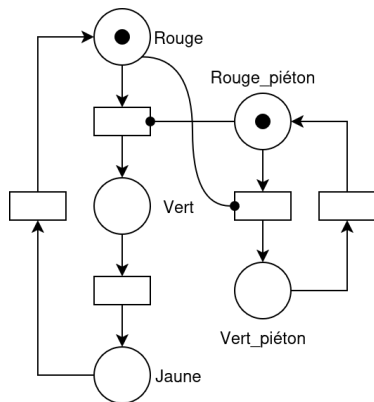
Attention à la combinaison de modalités !



- "Toujours, un jour il y aura un jeton dans *Rouge*"
 $\models \Box \Diamond (Rouge = 1)$
(infiniment souvent)

LTL – Exemple avec modalités

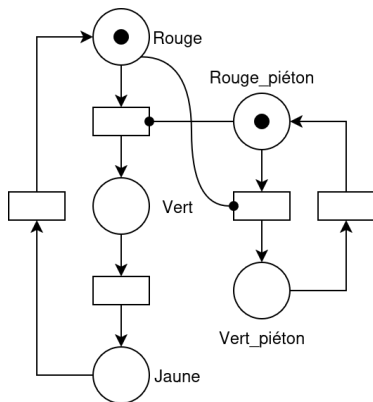
Attention à la combinaison de modalités !



- ▶ "Toujours, un jour il y aura un jeton dans *Rouge*"
 $\models \Box \Diamond (Rouge = 1)$
(infiniment souvent)
- ▶ "Un jour, toujours il y aura un jeton dans *Rouge_piéton*"
 $\not\models \Diamond \Box (Rouge_pieton = 1)$

LTL – Exemple avec modalités

Attention à la combinaison de modalités !



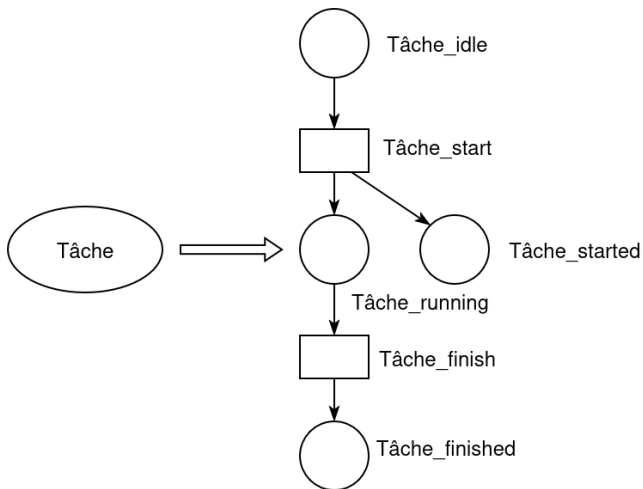
- ▶ "Toujours, un jour il y aura un jeton dans *Rouge*"
 $\models \Box \Diamond (Rouge = 1)$
(infiniment souvent)
- ▶ "Un jour, toujours il y aura un jeton dans *Rouge_piéton*"
 $\not\models \Diamond \Box (Rouge_pieton = 1)$
- ▶ "Toujours, si à un moment il y a un jeton dans *Vert*, alors dans le futur **à partir de ce moment** il y aura un jeton dans *Rouge*"
 $\models \Box (Vert = 1 \Rightarrow \Diamond (Rouge = 1))$

Note : syntaxe tina

Symbole	Tina	Nom
$\wedge$	$\wedge$	Conjonction
$\vee$	$\vee$	Disjonction
$\Rightarrow$	$\Rightarrow$	Implication
$\Leftrightarrow$	$\Leftrightarrow$	Équivalence
$\neg$	-	Négation
$\square$	[]	Toujours
$\diamond$	$\langle \rangle$	Un jour
$\geq$	$\geq$ ou ge	Supérieur ou égal
$>$	gt	Supérieur strict
$\leq$	$\leq$ ou le	Inférieur ou égal
$<$	lt	Inférieur strict

Note : Tina accepte la syntaxe A pour signifier $A > 0$

SIMPLEPDL vers PETRINET (rappel)



Retour à nos questions initiales

- ▶ "Une tâche est soit non commencée, soit en cours, soit terminée"

Retour à nos questions initiales

- ▶ "Une tâche est soit non commencée, soit en cours, soit terminée"
 $\square(A_started + A_running + A_finished = 1)$

Retour à nos questions initiales

- ▶ "Une tâche est soit non commencée, soit en cours, soit terminée"
 $\square(A_started + A_running + A_finished = 1)$
- ▶ "Une tâche commencée reste commencée pour toujours"

Retour à nos questions initiales

- ▶ "Une tâche est soit non commencée, soit en cours, soit terminée"
 $\Box(A_started + A_running + A_finished = 1)$
- ▶ "Une tâche commencée reste commencée pour toujours"
 $\Box(A_started \Rightarrow \Box A_started)$

Retour à nos questions initiales

- ▶ "Une tâche est soit non commencée, soit en cours, soit terminée"
 $\square(A_started + A_running + A_finished = 1)$
- ▶ "Une tâche commencée reste commencée pour toujours"
 $\square(A_started \Rightarrow \square A_started)$
- ▶ "Une tâche terminée reste terminée pour toujours"

Retour à nos questions initiales

- ▶ "Une tâche est soit non commencée, soit en cours, soit terminée"
 $\Box(A_started + A_running + A_finished = 1)$
- ▶ "Une tâche commencée reste commencée pour toujours"
 $\Box(A_started \Rightarrow \Box A_started)$
- ▶ "Une tâche terminée reste terminée pour toujours"
 $\Box(A_finished \Rightarrow \Box A_finished)$

Retour à nos questions initiales

- ▶ "Une tâche est soit non commencée, soit en cours, soit terminée"
 $\Box(A_started + A_running + A_finished = 1)$
- ▶ "Une tâche commencée reste commencée pour toujours"
 $\Box(A_started \Rightarrow \Box A_started)$
- ▶ "Une tâche terminée reste terminée pour toujours"
 $\Box(A_finished \Rightarrow \Box A_finished)$
- ▶ "Une tâche terminée a commencé un jour"

Retour à nos questions initiales

- ▶ "Une tâche est soit non commencée, soit en cours, soit terminée"
 $\square(A_started + A_running + A_finished = 1)$
- ▶ "Une tâche commencée reste commencée pour toujours"
 $\square(A_started \Rightarrow \square A_started)$
- ▶ "Une tâche terminée reste terminée pour toujours"
 $\square(A_finished \Rightarrow \square A_finished)$
- ▶ "Une tâche terminée a commencé un jour"
 $\square(A_finished \Rightarrow A_started)$

Retour à nos questions initiales

- ▶ "Une tâche est soit non commencée, soit en cours, soit terminée"
 $\Box(A_started + A_running + A_finished = 1)$
- ▶ "Une tâche commencée reste commencée pour toujours"
 $\Box(A_started \Rightarrow \Box A_started)$
- ▶ "Une tâche terminée reste terminée pour toujours"
 $\Box(A_finished \Rightarrow \Box A_finished)$
- ▶ "Une tâche terminée a commencé un jour"
 $\Box(A_finished \Rightarrow A_started)$
- ▶ "Initialement, aucune tâche n'est commencé"

Retour à nos questions initiales

- ▶ "Une tâche est soit non commencée, soit en cours, soit terminée"
 $\square(A_started + A_running + A_finished = 1)$
- ▶ "Une tâche commencée reste commencée pour toujours"
 $\square(A_started \Rightarrow \square A_started)$
- ▶ "Une tâche terminée reste terminée pour toujours"
 $\square(A_finished \Rightarrow \square A_finished)$
- ▶ "Une tâche terminée a commencé un jour"
 $\square(A_finished \Rightarrow A_started)$
- ▶ "Initialement, aucune tâche n'est commencé"
 $A_started = 0$

Retour à nos questions initiales

- ▶ "Une tâche est soit non commencée, soit en cours, soit terminée"
 $\square(A_started + A_running + A_finished = 1)$
- ▶ "Une tâche commencée reste commencée pour toujours"
 $\square(A_started \Rightarrow \square A_started)$
- ▶ "Une tâche terminée reste terminée pour toujours"
 $\square(A_finished \Rightarrow \square A_finished)$
- ▶ "Une tâche terminée a commencé un jour"
 $\square(A_finished \Rightarrow A_started)$
- ▶ "Initialement, aucune tâche n'est commencé"
 $A_started = 0$
- ▶ **"Un jour, la tâche sera terminée"**

Retour à nos questions initiales

- ▶ "Une tâche est soit non commencée, soit en cours, soit terminée"
 $\square(A_started + A_running + A_finished = 1)$
- ▶ "Une tâche commencée reste commencée pour toujours"
 $\square(A_started \Rightarrow \square A_started)$
- ▶ "Une tâche terminée reste terminée pour toujours"
 $\square(A_finished \Rightarrow \square A_finished)$
- ▶ "Une tâche terminée a commencé un jour"
 $\square(A_finished \Rightarrow A_started)$
- ▶ "Initialement, aucune tâche n'est commencé"
 $A_started = 0$
- ▶ **"Un jour, la tâche sera terminée"**
 $\diamond A_finished$

Sommaire

- 1 Introduction
- 2 Transformations modèle/texte (M2T, T2M)
- 3 Transformations modèle-à-modèle (M2M)
- 4 Transfert sémantique, validation de transformation
- 5 Conclusion

Conclusion

On a vu comment transformer un modèle en texte, un texte en modèle, un modèle en un autre modèle.

Conclusion

On a vu comment transformer un modèle en texte, un texte en modèle, un modèle en un autre modèle.

On peut valider une transformation en vérifiant des propriétés sur le modèle résultant, sous condition que le modèle en entrée soit correct

Conclusion

On a vu comment transformer un modèle en texte, un texte en modèle, un modèle en un autre modèle.

On peut valider une transformation en vérifiant des propriétés sur le modèle résultant, sous condition que le modèle en entrée soit correct
 $\Rightarrow$ *programmation par contrat, test des postconditions*

Conclusion

On a vu comment transformer un modèle en texte, un texte en modèle, un modèle en un autre modèle.

On peut valider une transformation en vérifiant des propriétés sur le modèle résultant, sous condition que le modèle en entrée soit correct
 $\Rightarrow$ *programmation par contrat, test des postconditions*

Si le formalisme le permet, on peut écrire des propriétés à valider qui sont la traduction de la sémantique du modèle initial

Conclusion

On a vu comment transformer un modèle en texte, un texte en modèle, un modèle en un autre modèle.

On peut valider une transformation en vérifiant des propriétés sur le modèle résultant, sous condition que le modèle en entrée soit correct
 $\Rightarrow$ *programmation par contrat, test des postconditions*

Si le formalisme le permet, on peut écrire des propriétés à valider qui sont la traduction de la sémantique du modèle initial
 $\Rightarrow$ *PetriNet avec LTL*

Conclusion

On a vu comment transformer un modèle en texte, un texte en modèle, un modèle en un autre modèle.

On peut valider une transformation en vérifiant des propriétés sur le modèle résultant, sous condition que le modèle en entrée soit correct
 $\Rightarrow$ *programmation par contrat, test des postconditions*

Si le formalisme le permet, on peut écrire des propriétés à valider qui sont la traduction de la sémantique du modèle initial
 $\Rightarrow$ *PetriNet avec LTL*

À noter que la LTL de Tina étant du texte, ça se génère aisément pour tout modèle avec une transformation M2T

Conclusion

On a vu comment transformer un modèle en texte, un texte en modèle, un modèle en un autre modèle.

On peut valider une transformation en vérifiant des propriétés sur le modèle résultant, sous condition que le modèle en entrée soit correct
 $\Rightarrow$ *programmation par contrat, test des postconditions*

Si le formalisme le permet, on peut écrire des propriétés à valider qui sont la traduction de la sémantique du modèle initial
 $\Rightarrow$ *PetriNet avec LTL*

À noter que la LTL de Tina étant du texte, ça se génère aisément pour tout modèle avec une transformation M2T
 $\Rightarrow$ *Avec toute transfo SIMPLEPDL $\Rightarrow$ PETRINET, on transforme aussi SIMPLEPDL $\Rightarrow$ LTL ("certificat de bonne transformation")*